

Artificial Intelligence

Homework 3

Prof. Truong-Son Hy

September 24, 2025

Deadline

October 1, 2025

Note: Remember to submit your code and report that includes your final results.

Problem 1 - 50 points

In the class, we have implemented a version of MLP. In this homework, you will update the code further. You have to submit **both the code and a written report**.

1. Update the implementation of MLP so that the user can create as many hidden layers as they want, they can decide the number of neurons in each hidden layer. You can use `nn.ModuleList` of PyTorch. Furthermore, the user can use any non-linearity / activation they want (e.g., sigmoid, tanh, ReLU, leaky ReLU, etc.). All these options must be available in the constructor of the MLP class.
2. Create option to use Adam and AdaGrad optimization methods. They are better than Stochastic Gradient Descent (SGD). You can check the PyTorch document for optimizers:

<https://pytorch.org/docs/stable/optim.html>

3. You should try to use Google Colab **with a GPU**. Google Colab offer free usage of GPU and you can run your IPython notebook there:

<https://colab.research.google.com/>

You have to convert the model and training to GPU with the help of `.device("cuda")`. If your local machine has a GPU then you can use your local machine.

4. Evaluate with the Fashion MNIST dataset. You should use the dataloader of PyTorch:

<https://pytorch.org/vision/0.15/datasets.html>

You should have a validation test separated from the test set. The validation set is to select the best trained model. You should report the accuracy on the test set with the best model. You should save the model (i.e. learned weights) to file.

In the report, you must write the results including the testing accuracy, a figure demonstrating the training and validation accuracy after each epoch, the confusion matrix for **both the MNIST and the Fashion MNIST**.

Problem 2 - 50 points

Learning Goals

- Construct a small synthetic dataset for the XOR function and create train/validation splits.
- Implement and train a two-layer (one hidden layer) multilayer perceptron (MLP) for binary classification.
- Explain why XOR is not linearly separable and how nonlinear hidden layers resolve this.
- Evaluate classification performance.

Background

The Boolean XOR function on two inputs $x_1, x_2 \in \{0, 1\}$ is defined by

$$y = x_1 \oplus x_2 = \begin{cases} 1 & \text{if } x_1 \neq x_2, \\ 0 & \text{if } x_1 = x_2. \end{cases}$$

The four noiseless corner points are $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$. No single linear decision boundary can separate the positive and negative classes, which makes XOR a canonical example requiring nonlinearity.

Instructions

Part A – Build a Small Dataset (XOR)

1. **Noiseless core (required).** Start with the four corner points above.
2. **Augmented samples (recommended).** Generate $n = 200$ additional samples by first choosing a corner uniformly at random and then adding i.i.d. Gaussian noise $\mathcal{N}(0, \sigma^2)$ to each coordinate, with $\sigma \in [0.05, 0.15]$. The label is inherited from the chosen corner.
3. **Split.** Create an 80%/20% train/validation split (stratify if convenient).
4. **Plot.** Produce a 2D scatter plot of all points colored by label.

Part B – Model: MLP ($2 \rightarrow H \rightarrow 1$)

1. **Architecture.** One hidden layer with width H (default $H = 4$; you may try $H \in \{2, 4, 8\}$). Use a nonlinear activation in the hidden layer (*e.g.*, tanh or ReLU). The output is a single logit converted to a probability for class 1.
2. **Loss & optimization.** Use binary cross-entropy loss and any first-order optimizer (*e.g.*, SGD or Adam). Reasonable learning rates are in $[10^{-3}, 10^{-1}]$. Train for 100–300 epochs or the equivalent number of steps.
3. **Target performance.** On the augmented dataset, aim for $\geq 95\%$ validation accuracy. (The noiseless core alone is solvable to 100%.)

Part C – Evaluation & Visualization

1. Report final training and validation accuracy.
2. Provide a concise learning-curve plot (loss or accuracy vs. epochs).

Part D – Short Written Answers (1–2 sentences each)

1. Why can a linear classifier not solve XOR?
2. How does adding a hidden layer with a nonlinearity enable solving XOR?
3. Compare tanh vs. ReLU for this task (convergence behavior, shape of boundary, stability).
4. What changes when $H = 1$ versus $H = 8$? Briefly explain in terms of model capacity and fit.
5. How does increasing input noise σ affect the optimal boundary and generalization?

Deliverables

Submit a single PDF (exported from your notebook or report) that includes:

1. Dataset description and scatter plot.
2. Model description (architecture, activation, H) and training details (optimizer, learning rate, epochs, batch size).
3. Results: final train/validation accuracy; learning curve.
4. Short answers (Part D).